ICS 604
Applied Data Science

Department of Information and Computer Sciences
University of Hawai`i at Mānoa

Kyungim Baek

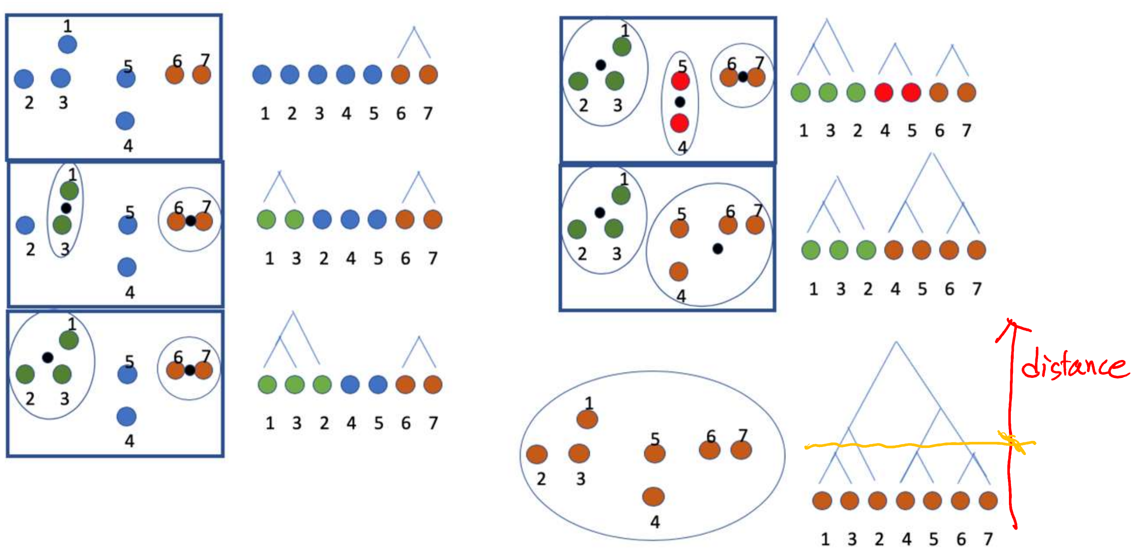
Reminder: Homework Assignment 4

- Due: **11:59 PM** on **Monday, April 27**
- Please carefully read and follow the assignment instructions and problem requirements.
- Submit only the Jupyter Notebook file (.ipynb) to Lamaku.
  - Do not upload the data file.
  - Make sure to **rename your notebook file** as instructed.

Lecture 25

- Hierarchical clustering
- K-means clustering
- Evaluating clustering results
- Mixture models

Previously… Hierarchical clustering



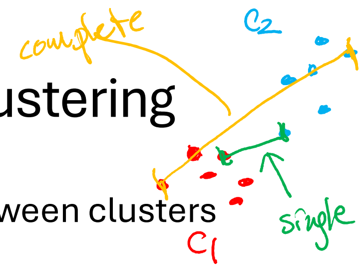
Hierarchical clustering (cont'd.)

- Repeatedly combine the two "closest" clusters into one.
- How do you know which pair of clusters is the closest?
  - We can compute the distances between two points.
- One possible solution:
  - Represent the location of each cluster using a single measure.
    - E.g., centroid, or the average of all points in a cluster
  - Take all centroids and find the closest pair.

Question

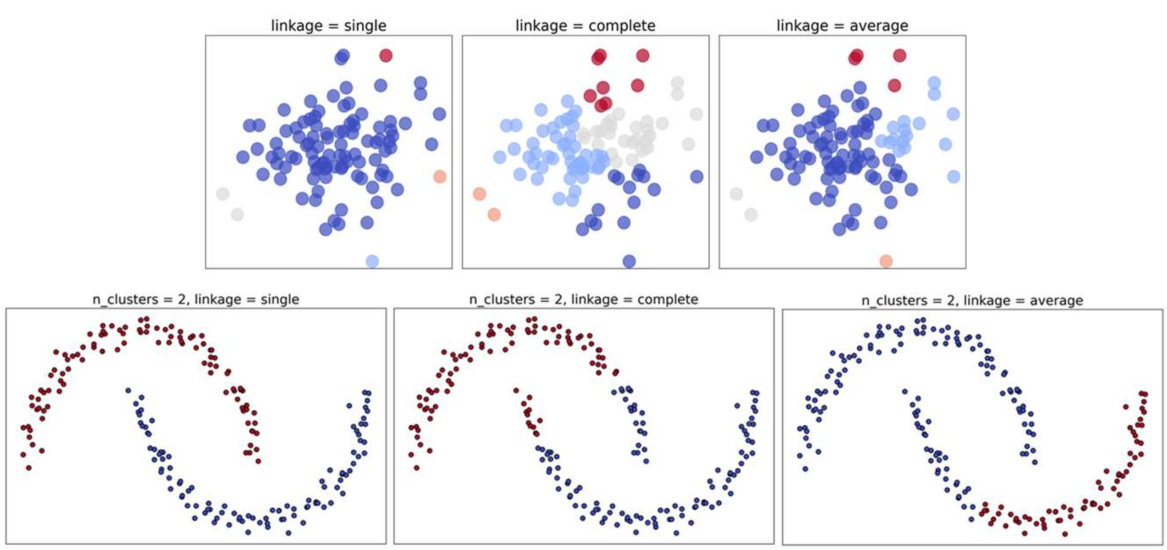
- Does the centroid approach work for all types of data?
  - For example, how do you compute the centroid for the DNA sequences?

Linkage criteria for hierarchical clustering

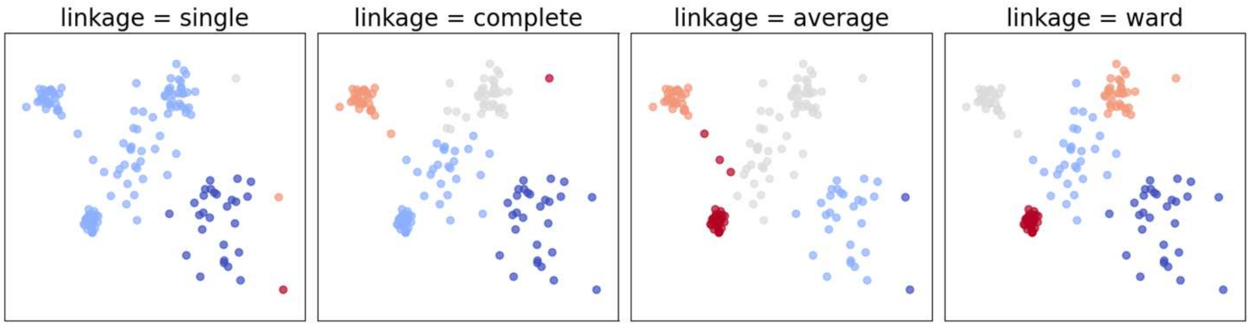


- We can define how to measure dissimilarity between clusters using different linkage criteria:
  - **Single linkage**: Minimum distances between any two points, one from each cluster
  - **Complete linkage**: Maximum distance between any two points, one from each cluster
  - **Average linkage**: Average distance between all pairs of points across the two clusters
- Variance-based criterion:
  - **Ward's method**: Merges clusters that result in the smallest increase in within-cluster variance

Examples of using different linkage criteria



Example of using different linkage criteria




```python
from sklearn.cluster import AgglomerativeClustering

for idx, linkage in enumerate(['single', 'complete', 'average', 'ward']):
    agg = AgglomerativeClustering(n_clusters=5, linkage=linkage)
    labels = agg.fit_predict(X)
    print(f"{linkage:8}: {np.bincount(labels)}")
```

```
single  : [ 27 120   1   1   1]
complete: [28 54 36 31  1]
average : [ 2 27 58 30 33]
ward    : [29 29 31 31 30]
```

Hierarchical clustering results

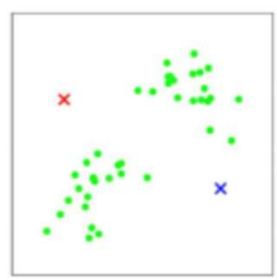
- Easy to implement and understand (plus).
    - Provides an interpretable visualization of the algorithm and data.
- Useful summarization tool giving a holistic view (plus).
- Method is computationally intensive (minus).
- Extracting meaningful clusters can be tricky! (minus).
    - It's not always obvious where to "cut" the tree (dendrogram) to form clusters.
        - The tree can have no clear big gaps to suggest natural divisions.
        - Different cut heights can produce wildly different clustering.

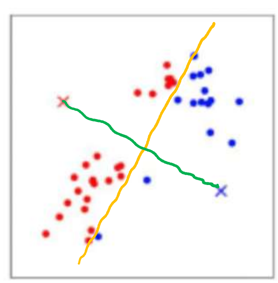
Assigning points to clusters: K-means clustering

- A popular clustering algorithm for partitioning data into a predefined number of groups.
- Steps:
  1. Define the number of clusters.
  2. Initialize cluster representatives (centroids).
  3. Assign points to each cluster.
  4. Recalculate the centroids.
  5. Repeat steps 3 and 4.

K-means clustering steps

1. Define the number of clusters k.
2. **Initialize** clusters by k points; one per cluster.
   - Those are the cluster representatives; the cluster centroids.
     - For instance, take k points in space at random.
3. **Assign points to clusters**
   - Place each point in the cluster identified by the closest centroid.
   - Here, we "paint" the examples with the same color as the cluster centroid to which it was assigned.

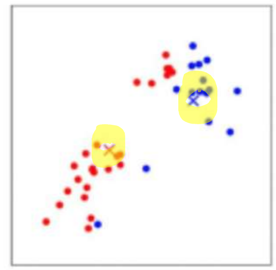




K-means clustering steps (cont'd.)

4. Recalculate the centroids

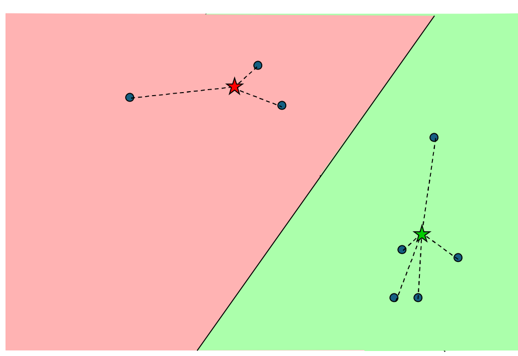
- After all points are assigned, recompute the centroids of the k clusters.



5. Assign and recalculate centroids

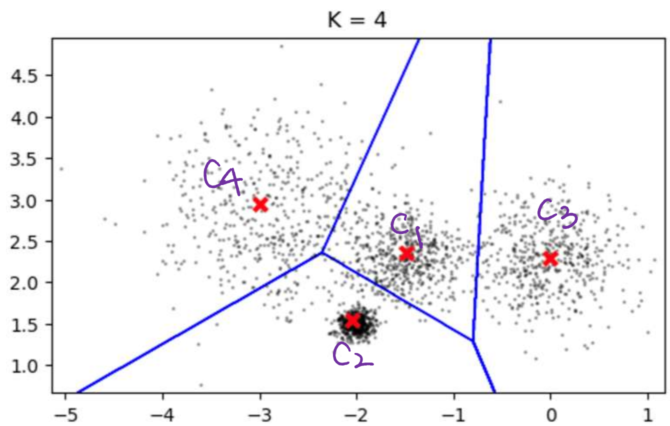
- Repeat steps 3 and 4 until convergence.
  - The clustering is stable, and nothing changes.
    - I.e., assigning the points to the closest centroid and recomputing the centroids does not change the centroids or the assignments.

K-means clustering in action



Slide credit: S. Thrun

K-means clustering results



Inertia: sum of squared distances of samples to their closest cluster center (value of the objective function)

```
from sklearn.cluster import KMeans

kmeans = KMeans(n_clusters=4, n_init=10)
kmeans.fit_predict(X)

print(kmeans.cluster_centers_)
print(kmeans.labels_)
```

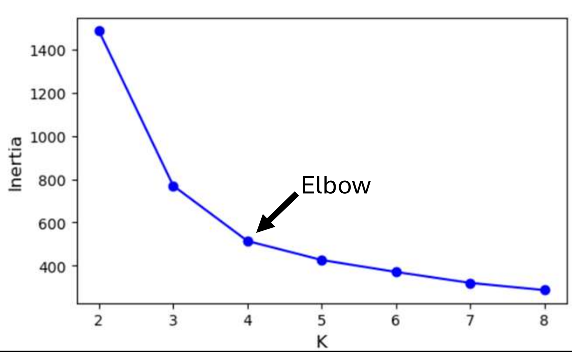
```
[[-1.48574718  2.36569662] C1
 [-2.0478565   1.54876933] C2
 [-0.00456774  2.29207952] C3
 [-2.99080454  2.9500348 ]] C4
[2 0 0 ... 2 2 0]
```

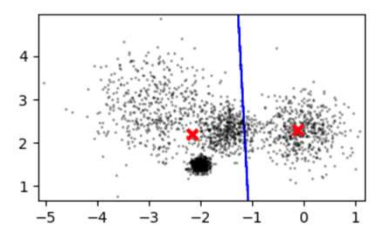
```
print(kmeans.inertia_)
print(kmeans.score(X))
```

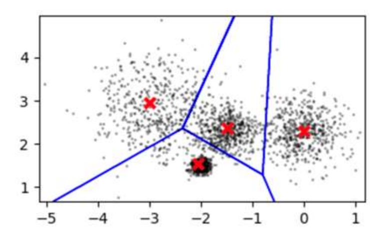
```
513.8461272445492
-513.8461272445492
```

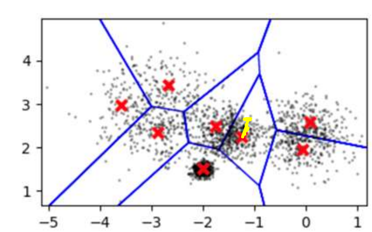
Inertia against K

- Inertia (within-cluster sum-of squares) is the sum of squared distances of each point to its assigned cluster center.



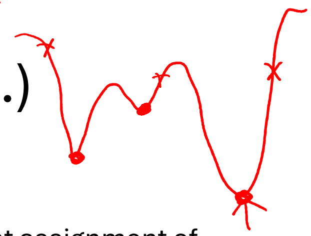

K = 2


K = 4


K = 8

obje.

K-means clustering results (cont'd.)



- Algorithm is non-deterministic (minus).
  - A different choice of starting values may result in a different assignment of points to clusters.
    - Customary to run the k-means algorithm several times and compare results or compute a consensus.
- Cannot be used with categorical data (non-Euclidean space) (minus).
  - Difficult, or sometimes impossible, to construct centroids from categorical data.
- Can use k-medoid, the smallest average distance to all other points in the cluster (plus).
  - A medoid is the point in the cluster with the smallest distance to all the other points in the cluster.
    - Less affected by outliers; more stable.

K-means clustering results (cont'd.)

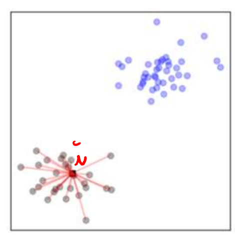
- Small number of iterations to converge (10–50 iterations are typical) (plus).
- Can take a long time to converge with some poor initializations (minus).
  - Very rare, but a possibility, nevertheless.
- Works relatively well in real life.

How do we evaluate the results of clustering

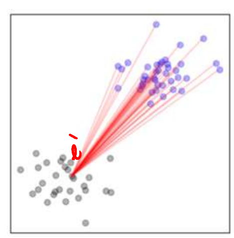
- What distinguishes a good clustering solution from a bad one?
- Recall that what we want is:
  1. Points within the same cluster to be very similar
     - The average distance between points in the same cluster should be small (**high *cohesion***).
  2. Points across clusters to be highly dissimilar
     - The distance between points from distinct clusters should be large (**high *separation***).
- A criteria to evaluate a clustering solution: $\frac{\boldsymbol{separation}}{\boldsymbol{cohesion}}$
  - Large separation and high cohesion (small average within-cluster distance).
  - Large values are indicative of a good separation.

The silhouette coefficient

- A good "score" to estimate the ratio of separation to cohesion.
- For each cluster we denote:
  - a_i is the average distance that point i has from all points in the same cluster (i.e., the mean intra-cluster distance).
    - Represents the cohesion; we want a_i to be as small as possible.



  - b_i be the smallest (or average) distance between point i and a point not in the same cluster as point i.
    - Represents separation; we want b_i to be as large as possible.



The silhouette coefficient (cont'd.)

- We measure the silhouette coefficient of a point i as

$$S_i = \frac{b_i - a_i}{\max(a_i, b_i)}$$

- Silhouette coefficient ranges from −1 to 1.
  - Negative values indicate a_i is larger than b_i.
    - Low cohesion or small separation or both.
    - This suggests poor clustering.
  - Positive values indicate b_i is larger than a_i.
    - High cohesion or high separation or both.

```python
def compute_a(cluster, target_pt_id):
    distances = []

    for other_pt_id in range(len(cluster)):
        if target_pt_id != other_pt_id:
            distances.append(distance.euclidean(cluster[target_pt_id],
                                                cluster[other_pt_id]))

    return np.mean(distances)
```

```python
### Use minimum distance

from scipy.spatial import distance

def compute_b(pt_cl1, c2):
    min_dist = np.inf

    for j, pt2 in enumerate(c2):
        d = distance.euclidean(pt_cl1, pt2)
        if d < min_dist:
            min_dist = d

    return min_dist
```

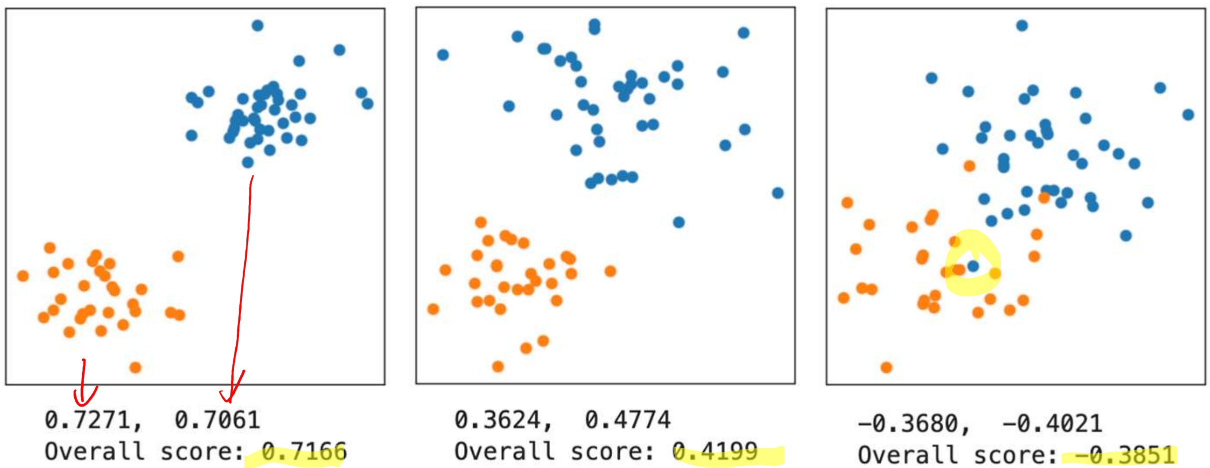
```python
def pt_silhouette(cluster, target_pt_id, other_cluster):
    a_i = compute_a(cluster, target_pt_id)
    b_i = compute_b(cluster[target_pt_id], other_cluster)

    return (b_i - a_i) / max(a_i, b_i)
```

Computing cluster-wide silhouette coefficient

- Average silhouette coefficients within cluster.
  - A good indicator of the quality of an individual cluster.
  - As before, larger values indicate better separation and more coherent clusters.
- **Silhouette score:** Average over the silhouette coefficients for all data points.
  - Provides a good estimate of the quality of the overall clustering solution.
  - Higher values indicate, on average, better cohesion within clusters and better separation between clusters.

```python
# For simplicity, the solution assumes two clusters
# Need to consider additional clusters in solution

def cluster_silhouette(cluster, other_cluster):
    silhouette_coeffs = []
    for pt_id in range(len(cluster)):
        silhouette_coeffs.append(pt_silhouette(cluster, pt_id, other_cluster))

    return(np.mean(silhouette_coeffs))
```

```python
avg_sil_coef_c1 = cluster_silhouette(c1, c2)
avg_sil_coef_c2 = cluster_silhouette(c2, c1)

print(f"{avg_sil_coef_c1:.4f},  {avg_sil_coef_c2:.4f}")
print(f"Overall score: {np.mean([avg_sil_coef_c1, avg_sil_coef_c2]):.4f}")
```

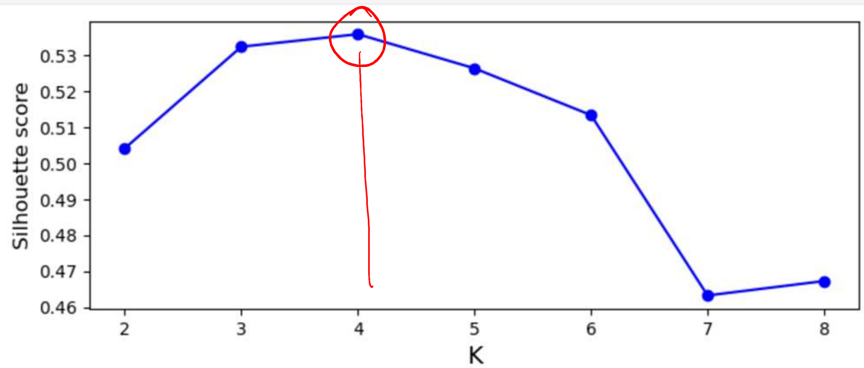


Using silhouettes to decide on best number of cluster

- Silhouette score can be useful to determine the number of clusters present in the dataset.
- Run the algorithm several times for each possible value of k, and compute the overall silhouette score each time.
  - We should observe a peak at the best value of k.

- Question:
  - For k-means, why do we need to run it *several times* for each value of k we would like to test?

```python
from sklearn.metrics import silhouette_score

s_scores = [silhouette_score(X, model.labels_) for model in kmeans_per_k]

plt.figure(figsize=(8, 3))
plt.plot(range(2, 9), s_scores, "bo-")
plt.xlabel("K", fontsize=14)
plt.ylabel("Silhouette score", fontsize=12)
plt.show()
```



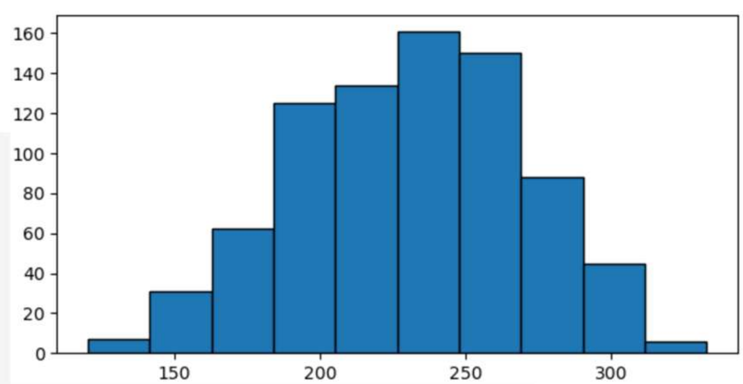
Mixture Models

Mixture model intuition

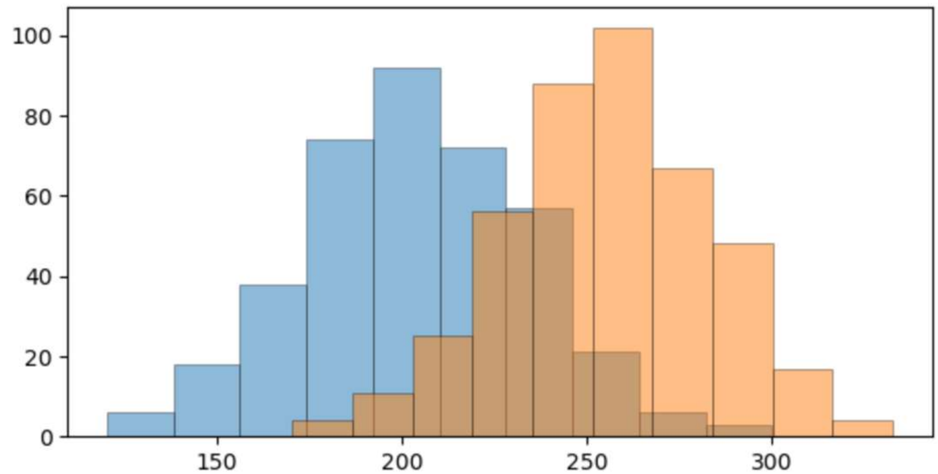
Example:

- Knowing that there is a difference between the spending of male and female purchases on a platform and given the sales data where males and females are not labeled, we can use k-means to find the clusters in the data.
- We can also model the spending as Gaussians and try to find the mean and the standard deviation of each distribution.
  - Each data point is simply a random variate from a Gaussian.
  - A data point can have a probability density value under each distribution.
- Let's begin by presuming that we can identify the person associated with each data point.

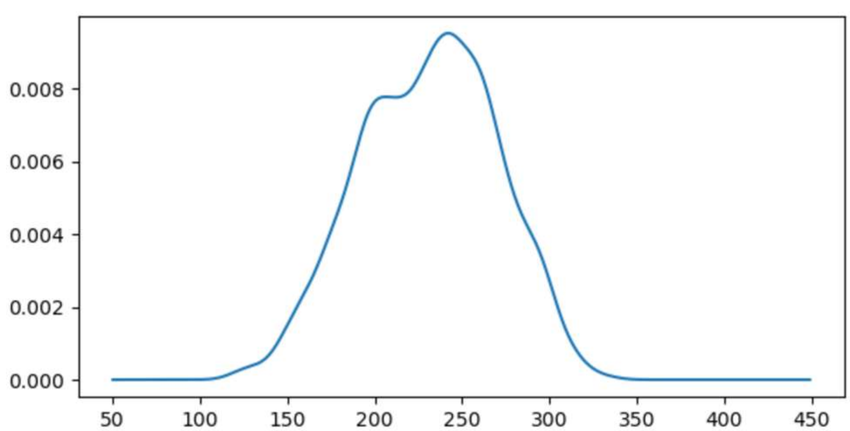
```
np.random.seed(45)

mean_males = 204
std_males = 31

mean_females = 257
std_females = 28

sales_males = np.random.normal(mean_males, std_males, 387)
sales_females = np.random.normal(mean_females, std_females, 422)

sales_total = np.concatenate([sales_males, sales_females])

plt.hist(sales_total, edgecolor='k', linewidth=1);
```



```
plt.hist(sales_males, alpha=0.5, edgecolor='k', linewidth=0.5)
plt.hist(sales_females, alpha=0.5, edgecolor='k', linewidth=0.5);
```



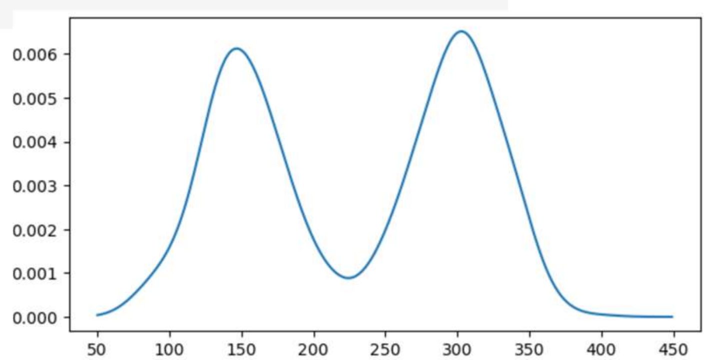

```
kde_total = sp.stats.gaussian_kde(sales_total, bw_method=0.2)

x_axis = np.arange(50, 450)
y_axis_total = kde_total.evaluate(x_axis)
plt.plot(x_axis, y_axis_total);
```



Finite mixture model

- The distribution exhibits two different peaks.
  - We know that the total sales distribution is a mixture of precisely two distributions.
  - Those are clearer in the KDE than in the histogram.
  - They would be much more evident if the means of the two distributions were much farther away.

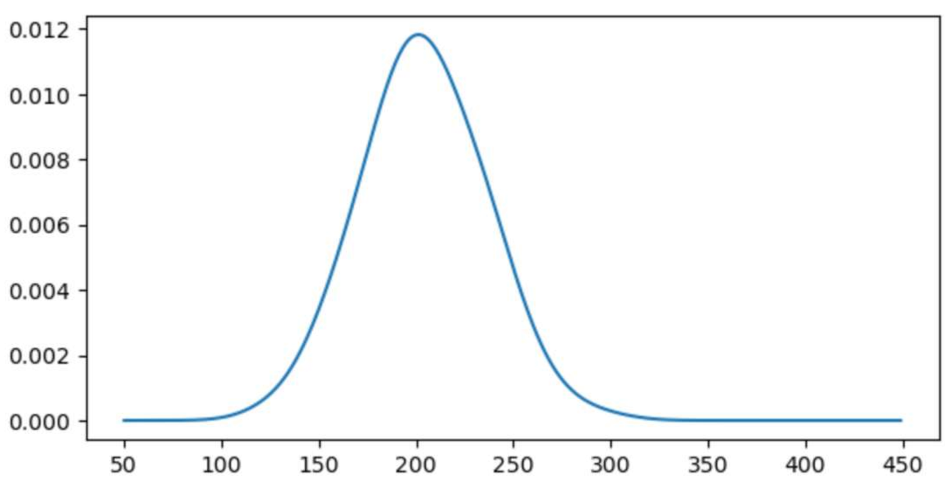
```
mean_males2, std_males2 = 150, 31
mean_females2, std_females2 = 300, 28

sales_males2 = np.random.normal(mean_males2, std_males2, 387)
sales_females2 = np.random.normal(mean_females2, std_females2, 422)

sales_total2 = np.concatenate([sales_males2, sales_females2])

kde_total2 = sp.stats.gaussian_kde(sales_total2, bw_method=0.17)
y_axis2 = kde_total2.evaluate(x_axis)
plt.plot(x_axis, y_axis2);
```



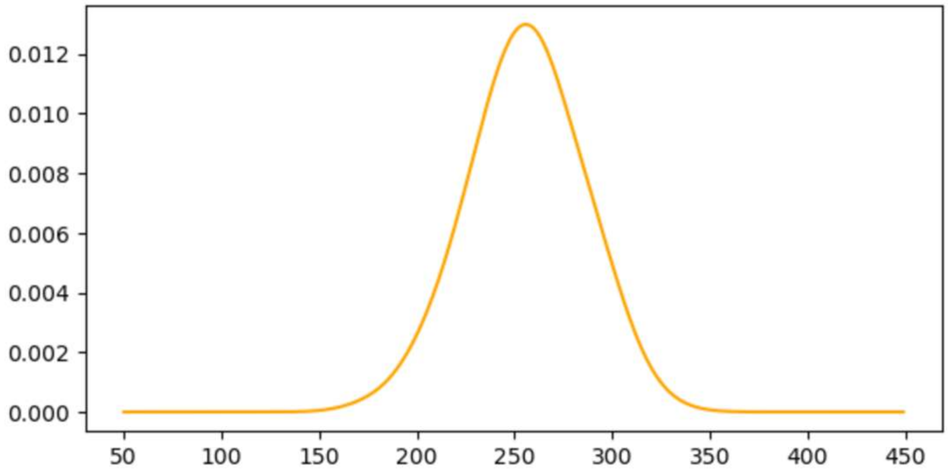
Finite mixture model (cont'd.)

- Here, the total sales are represented as a finite mixture of two distributions.
  - A mixture distribution is the probability distribution of a random variable derived from a collection of other random variables.
    - i.e., a combined probability distribution that arises from blending several other distributions.
- Mixtures occur naturally. For example:
  - When analyzing spending on the Apple Store, the distribution of prices for professional applications may be substantially different from those for the general public.
    - For instance, prices for apps designed for doctors, stock traders, and frequent travelers may significantly differ from those intended for students.
  - Recovery time for patients may vary drastically depending on various factors such as the type of treatment or the patient's pre-existing conditions.
    - May lead to people who recover quickly and those who don't.
- We can think of these mixture components as clusters.

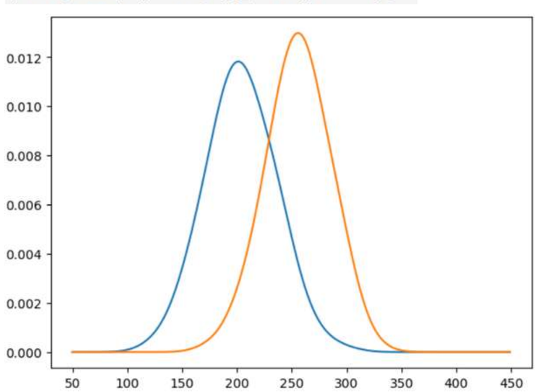
```
kde_males = sp.stats.gaussian_kde(sales_males, bw_method=0.5)
y_axis_males = kde_males.evaluate(x_axis)
plt.plot(x_axis, y_axis_males);
```

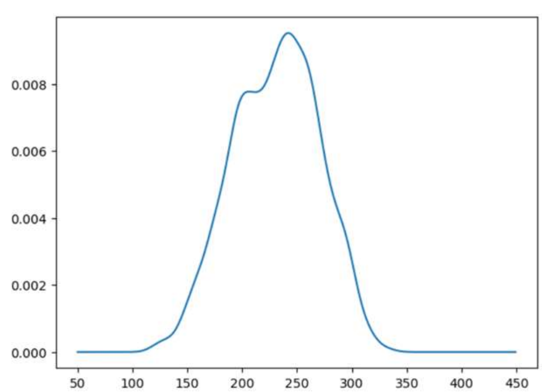


```
kde_females = sp.stats.gaussian_kde(sales_females, bw_method=0.5)
y_axis_females = kde_females.evaluate(x_axis)
plt.plot(x_axis, y_axis_females, color='orange');
```

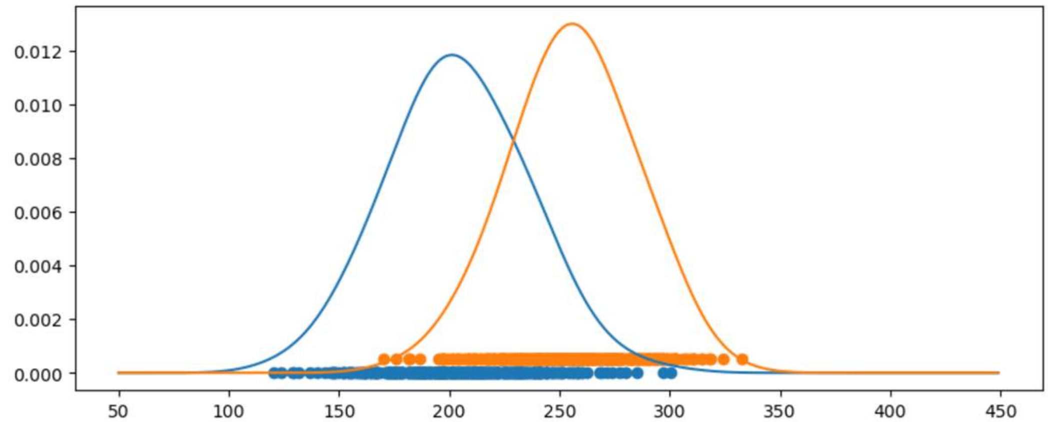



```
plt.figure(figsize=(15, 5))

plt.subplot(1, 2, 1)
plt.plot(x_axis, y_axis_males)
plt.plot(x_axis, y_axis_females)

plt.subplot(1, 2, 2)
plt.plot(x_axis, y_axis_total);
```





```python
plt.figure(figsize=(10, 4))

plt.plot(x_axis, y_axis_males)
plt.plot(x_axis, y_axis_females)

plt.scatter(sales_males, np.zeros(len(sales_males)))
plt.scatter(sales_females, np.zeros(len(sales_females)) + 0.0005);
```



Using k-means to identify the two clusters

- We know the `sales_total` is a concatenation of `sales_males` and `sales_females`.
  - Contains the sales information from 387 males and 422 females.
- We can assign each entry in `sales_total` either male (0) or female (1).

```python
y_true = np.concatenate([np.zeros(387), np.ones(422)])
y_true[380:394]
```

```
array([0., 0., 0., 0., 0., 0., 0., 1., 1., 1., 1., 1., 1., 1.])
```

```python
from sklearn.cluster import KMeans

X = sales_total.reshape(-1, 1)

kmeans = KMeans(n_clusters=2, n_init=5).fit(X)
y_predicted = kmeans.predict(X)

kmeans.cluster_centers_
```

true

```
array([[259.58191241],
       [194.81303572]])
```

female : 257

male : 204

```python
y_predicted[0:30]
```

```
array([1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 0, 1, 1, 0, 1])
```

```python
y_predicted[-30:]
```

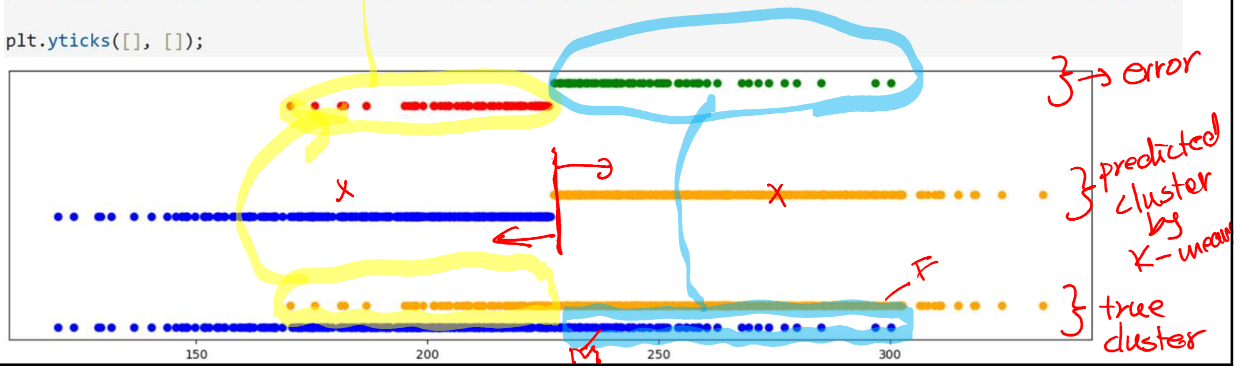
```
array([1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0,
       1, 1, 0, 0, 0, 0, 0, 0])
```

```python
y_predicted = 1 - y_predicted
sum(y_predicted == y_true) / len(y_predicted)
```

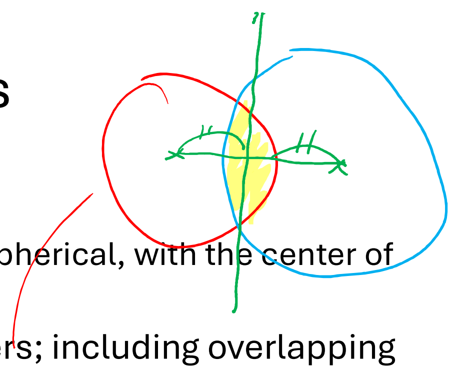
```
0.8108776266996292
```

```python
pred_males = np.where(y_predicted == 0)
pred_females = np.where(y_predicted == 1)

pred_males_error = np.where((y_predicted == 0) & (y_true == 1))
pred_females_error = np.where((y_predicted == 1) & (y_true == 0))
```

```
plt.figure(figsize=(16, 4))

# REAL DATA
plt.scatter(sales_total[:len(sales_males)], np.zeros(len(sales_males)), color='blue')
plt.scatter(sales_total[len(sales_males):], np.zeros(len(sales_females))+0.01, color='orange')

# PREDICTED DATA
plt.scatter(sales_total[pred_males], np.zeros(len(y_predicted[pred_males]))+0.05, color='blue')
plt.scatter(sales_total[pred_females], np.zeros(len(y_predicted[pred_females]))+0.06, color='orange')

# ERRORS IN PREDICTION
plt.scatter(sales_total[pred_males_error], np.zeros(len(y_predicted[pred_males_error]))+0.1, color='red')
plt.scatter(sales_total[pred_females_error], np.zeros(len(y_predicted[pred_females_error]))+0.11, color='green')

plt.yticks([], []);
```



Impracticality of the k-means

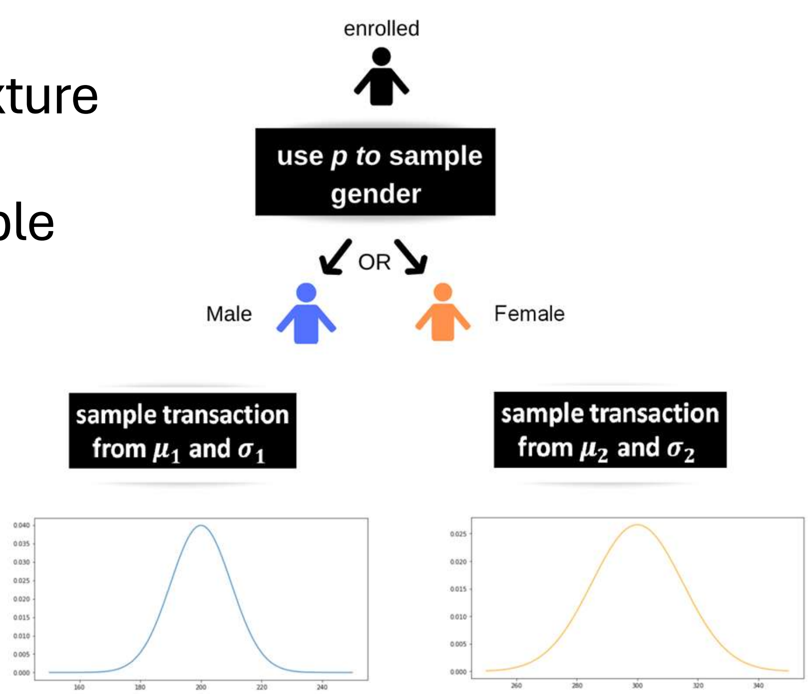


- Works best with blobs of data.
  - The boundary of each cluster is always spherical, with the center of the cluster being the mean.
- Does not work well on overlapping clusters; including overlapping blobs.
  - No confidence in the uncertainty in the assignment of each point.
- Here, we need an approach that let us include the uncertainty in the clustering.

Modeling a mixture distribution

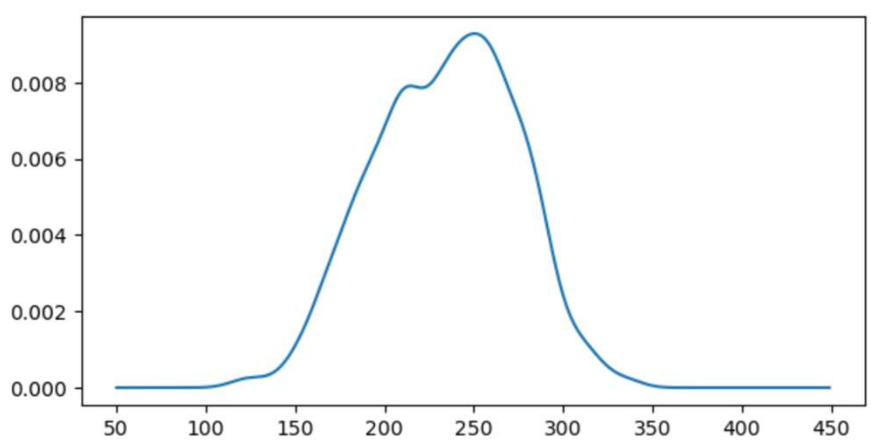
- In the earlier example, we assumed that we know the exact number of males and females.
  - The number of males and females should also be a parameter to our model.
    - E.g., sample gender from a binomial (ex. for 500 individuals and a proportion of males (or females) = 0.7).
- A more accurate generative model of the mixture distribution can be derived as follows.

Defining the mixture distribution for spending example

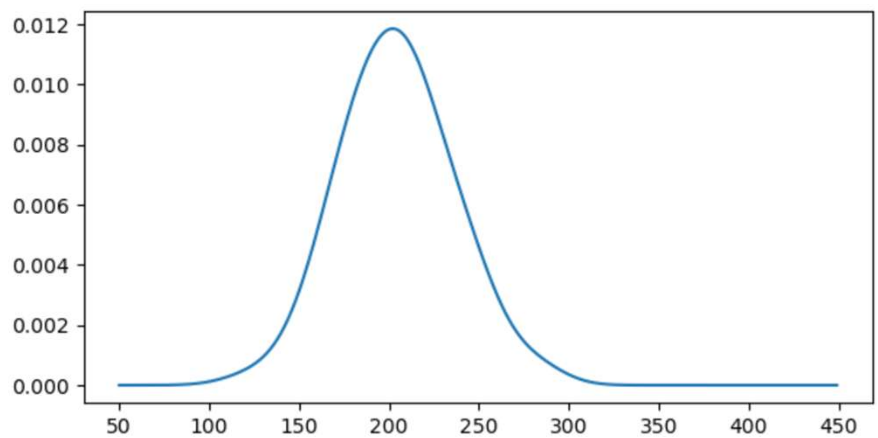


```python
np.random.seed(42)
male_female_prop = 0.45

mean_males = 204
std_males = 31

mean_females = 257
std_females = 28

sales_males =  []
sales_females = []
cluster = []

for i in range(1000):
    gender = np.random.choice([0, 1], p=[male_female_prop, 1 - male_female_prop])
    if gender == 0:
        cluster.append(0)
        male_sale = np.random.normal(mean_males, std_males)
        sales_males.append(male_sale)
    else:
        cluster.append(1)
        female_sale = np.random.normal(mean_females, std_females)
        sales_females.append(female_sale)

sales_total = np.concatenate([sales_males, sales_females])
```

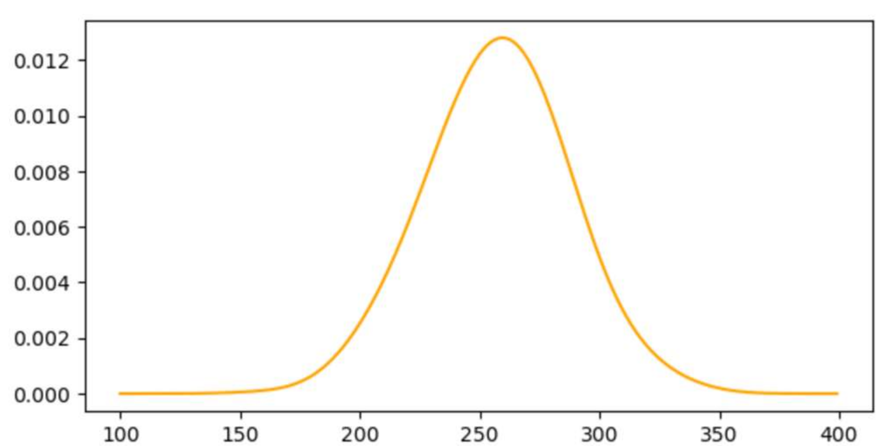
```python
kde_total = sp.stats.gaussian_kde(sales_total, bw_method=0.2)

x_axis_total = np.arange(50, 450)
y_axis_total_kde = kde_total.evaluate(x_axis_total)

plt.plot(x_axis, y_axis_total_kde);
```



```
kde_males = sp.stats.gaussian_kde(sales_males, bw_method=0.5)

x_axis_males = np.arange(50, 450)
y_axis_males_kde = kde_males.evaluate(x_axis_males)

plt.plot(x_axis_males, y_axis_males_kde);
```




```
kde_females = sp.stats.gaussian_kde(sales_females, bw_method=0.5)

x_axis_females = np.arange(100, 400)
y_axis_females_kde = kde_females.evaluate(x_axis_females)

plt.plot(x_axis_females, y_axis_females_kde, color="orange");
```



```
plt.figure(figsize=(15, 5))

plt.subplot(1, 2, 1)
plt.plot(x_axis_males, y_axis_males_kde)
plt.plot(x_axis_females, y_axis_females_kde)

plt.subplot(1, 2, 2)
plt.plot(x_axis, y_axis_total_kde);
```